

3D Solar System Simulation

Project Documentation

Course: Computer Graphics and Image Processing

Department: Computer Science and Engineering (CSE)

University: USTC

Semester: 7th Semester

Team Size: 2 Members

Technology: C++ | OpenGL | GLUT/GLFW

Table of Contents

1. [Project Overview](#)
 2. [Uniqueness & Why This Project](#)
 3. [Objectives](#)
 4. [CG Course Topics Covered](#)
 5. [Features List](#)
 6. [System Architecture](#)
 7. [Project Plan & Timeline](#)
 8. [Work Division](#)
 9. [Folder Structure](#)
 10. [Module Breakdown](#)
 11. [Tech Stack & Tools](#)
 12. [Controls & Interaction](#)
 13. [Expected Output](#)
 14. [References](#)
-

1. Project Overview

Project Title: 3D Interactive Solar System Simulation

Short Description:

A real-time 3D simulation of our Solar System built using OpenGL and C++. The simulation renders the Sun, all 8 planets, Earth's Moon, asteroid belt, and a star-filled background — all with proper lighting, texture mapping, orbital mechanics, and interactive camera controls.

This is not just a visual demo. It is a **scientifically inspired, graphically rich, interactive simulation** that demonstrates the core principles of Computer Graphics in a meaningful and visually impressive way.

2. Uniqueness & Why This Project

What makes this project UNIQUE?

Factor	Details
 Real-world Relevance	Based on actual planetary data (relative sizes, orbital speeds)
 Interactive	User can explore the solar system freely using keyboard & mouse
 Full CG Pipeline	Covers the ENTIRE Computer Graphics pipeline from scratch
 Educational Value	Can be used as a learning/teaching tool
 Visually Rich	Textures, lighting, shading, glow effects — all present
 Intelligent Design	Modular, scalable, clean architecture
 Impressive Presentation	Professors and audiences are always impressed by space simulations

Why choose THIS over other projects?

- A **game** like Pong or Breakout covers only 2-3 CG topics
- A **paint app** is more about UI than graphics
- A **Solar System Simulation** covers **every major CG topic** in one project
- It has **real-life scientific backing** — NASA, ESA use similar simulations
- It is **visually stunning** — perfect for project presentation/viva
- **2 members can divide work cleanly** without overlap

3. Objectives

1. Simulate the Solar System in a 3D OpenGL environment
2. Apply core CG concepts: transformations, lighting, shading, textures
3. Implement real-time animation with correct orbital mechanics
4. Provide interactive camera control for user exploration
5. Display planet information on screen (name, distance, speed)
6. Demonstrate mastery of the full Computer Graphics pipeline

4. CG Course Topics Covered

CG Topic	How It Is Used In This Project
2D & 3D Transformations	Planet rotation (self-axis) & revolution (around Sun)
Projection (Perspective)	Converting 3D world to 2D screen using perspective projection
Viewing & Camera	gluLookAt(), camera zoom, pan, orbit around solar system
Lighting Model	Sun acts as a POINT LIGHT SOURCE illuminating all planets
Phong Shading	Ambient + Diffuse + Specular shading on planet surfaces
Texture Mapping	Real planet surface textures mapped onto spheres
Animation	Continuous frame-by-frame planet movement
Z-Buffering (Depth Test)	Correct depth rendering — no overlap issues

CG Topic	How It Is Used In This Project
Color & Blending	Sun glow effect, transparent orbit rings
Geometric Primitives	Spheres (planets), lines (orbits), points (stars)
Coordinate Systems	World space, object space, camera space transformations
Display List / VBO	Efficient rendering of repeated objects

5. Features List

Core Features

- Sun at the center with glowing light emission effect
- All 8 planets: Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune
- Each planet has:
 - Unique size (proportionally scaled)
 - Unique orbital radius
 - Unique revolution speed (faster near Sun, slower far away)
 - Unique self-rotation speed
 - Real surface texture
- Earth's Moon orbiting Earth
- Saturn's Ring rendered using a flat torus/disc
- Asteroid Belt between Mars and Jupiter (particle-based)
- Star background (randomly placed point stars)
- Visible orbit paths (circular lines)

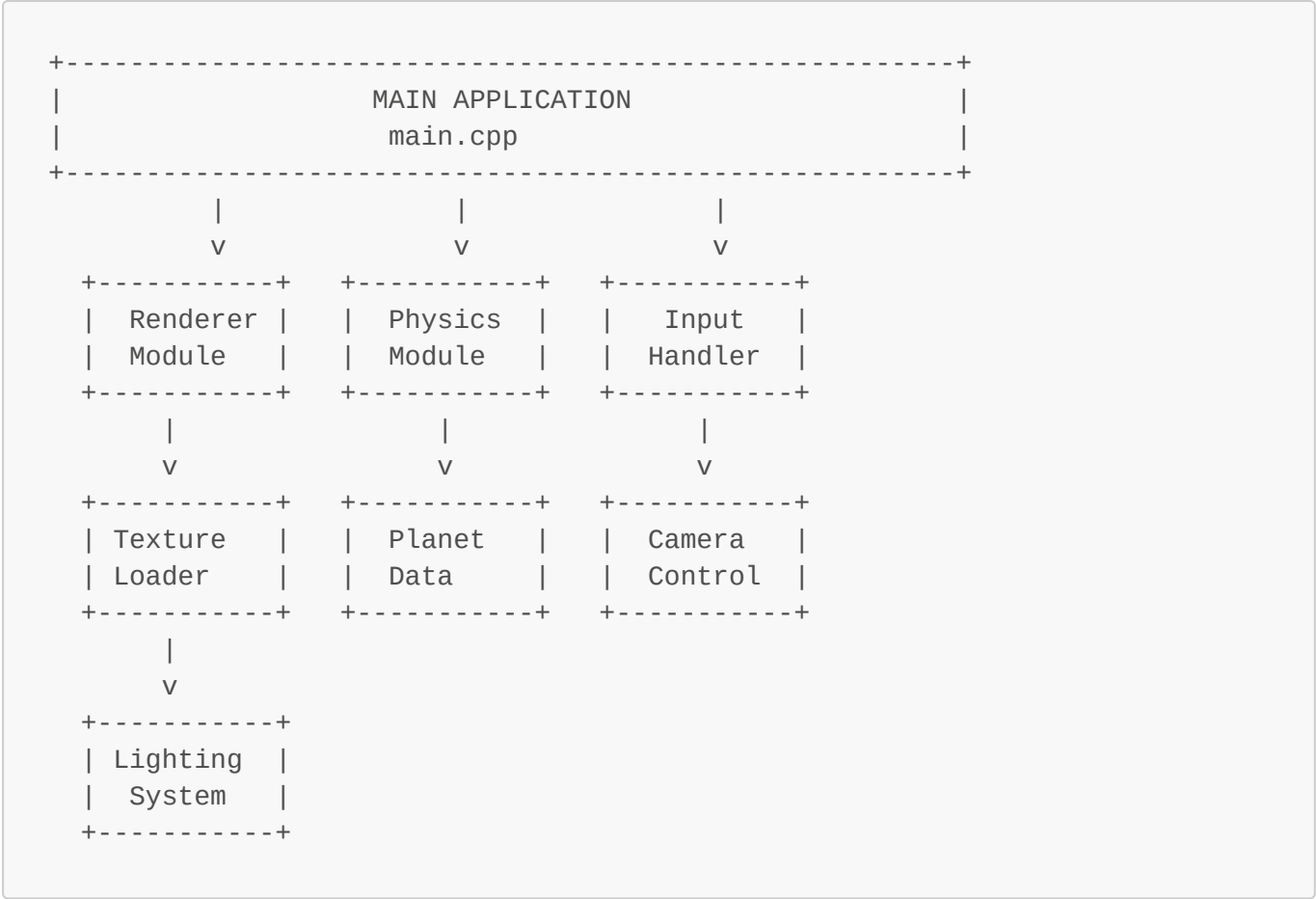
Interactive Features

- Camera rotation using mouse drag
- Zoom in / Zoom out using scroll wheel or keyboard
- Pause / Resume animation (SPACE key)
- Speed control — increase/decrease simulation speed
- Click on a planet → show planet info panel
- Toggle orbit lines ON/OFF
- Toggle planet labels ON/OFF
- Reset camera to default view

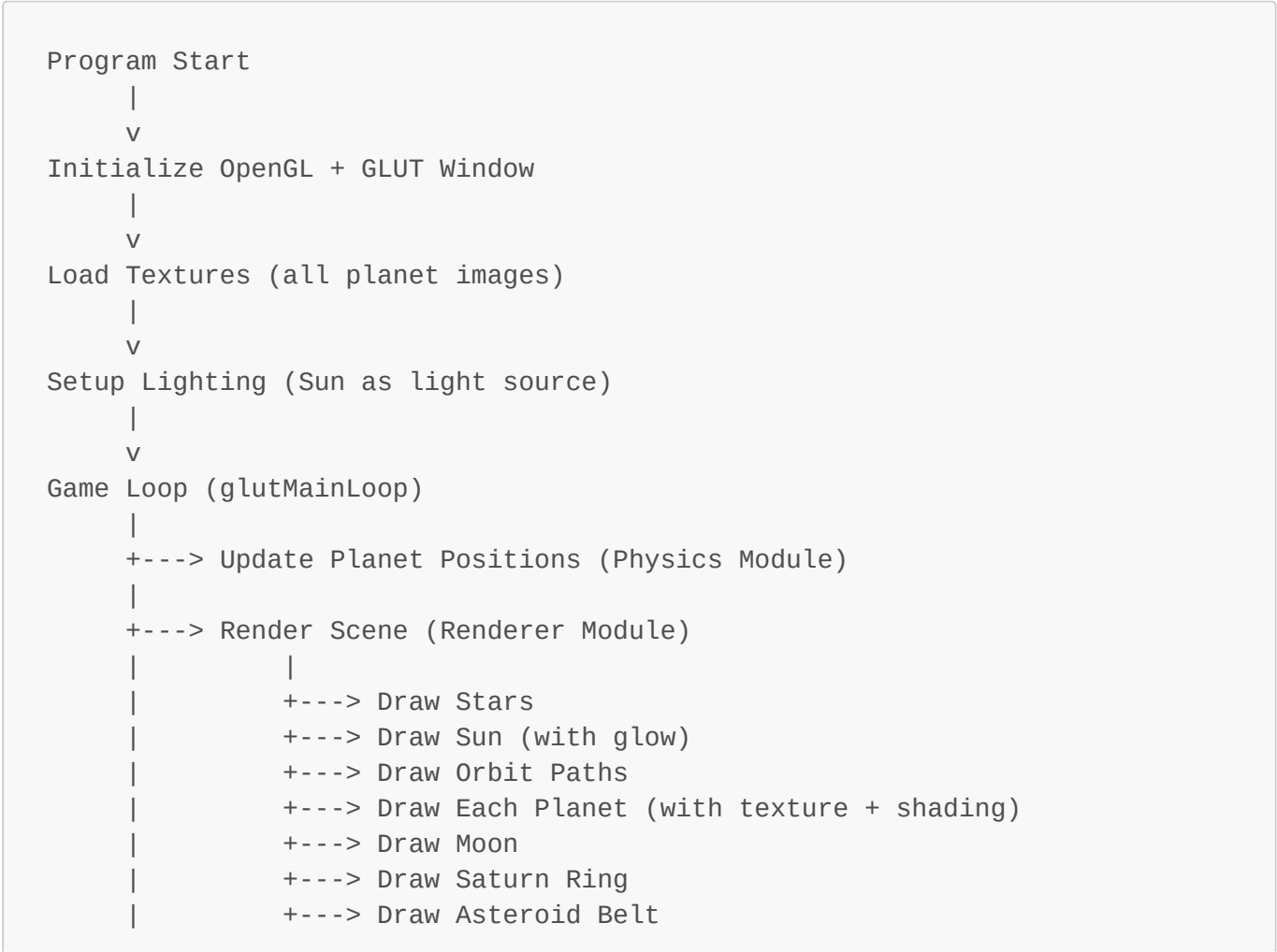
Visual Effects

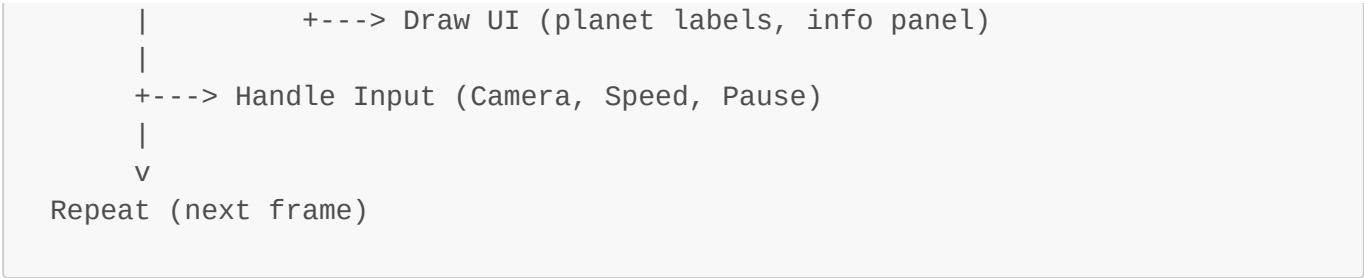
- Sun glow (light bloom simulation)
 - Shadow effect on planet dark side
 - Specular highlight on planet surface
 - Smooth animation (60 FPS target)
-

6. System Architecture



Data Flow





7. Project Plan & Timeline

Total Duration: 6 Weeks

Week	Tasks	Member
Week 1	Setup environment, OpenGL window, basic sphere rendering	Both
Week 2	Sun + planets with color, orbit animation (no texture yet)	Member 1
Week 2	Camera control (mouse + keyboard), zoom	Member 2
Week 3	Texture loading and mapping on all planets	Member 1
Week 3	Lighting setup (Sun as point light), Phong shading	Member 2
Week 4	Moon, Saturn ring, asteroid belt	Member 1
Week 4	Planet info UI, labels, orbit toggle	Member 2
Week 5	Visual effects (glow, star background, smooth animation)	Both
Week 6	Testing, bug fixing, documentation, presentation prep	Both

8. Work Division

Member 1 — Graphics & Rendering

- OpenGL window setup
- Planet & Sun sphere rendering
- Texture loading (stb_image.h)
- Texture mapping on spheres
- Saturn ring rendering
- Asteroid belt (particle system)
- Star background
- Final visual polish

Member 2 — Animation, Physics & Interaction

- Planet orbital mechanics (revolution angles, speeds)
- Planet self-rotation
- Moon orbit around Earth
- Camera system (gluLookAt, mouse drag, zoom)
- Keyboard input handling

- Planet info UI panel (glut text rendering)
- Pause/Resume, speed control
- Orbit path lines toggle

9. Folder Structure

```
Graphics_Project/
├── main.cpp                # Entry point
├── src/
│   ├── renderer.cpp        # All rendering functions
│   ├── renderer.h
│   ├── planet.cpp          # Planet class & data
│   ├── planet.h
│   ├── camera.cpp          # Camera control
│   ├── camera.h
│   ├── lighting.cpp        # Lighting setup
│   ├── lighting.h
│   ├── texture.cpp         # Texture loading
│   ├── texture.h
│   ├── ui.cpp              # On-screen text & info panel
│   └── ui.h
├── textures/
│   ├── sun.jpg
│   ├── mercury.jpg
│   ├── venus.jpg
│   ├── earth.jpg
│   ├── mars.jpg
│   ├── jupiter.jpg
│   ├── saturn.jpg
│   ├── uranus.jpg
│   ├── neptune.jpg
│   ├── moon.jpg
│   └── stars.jpg           # Background star texture
├── docs/
│   └── PROJECT_DOCUMENTATION.md
└── README.md
```

10. Module Breakdown

Module 1: Planet Data (planet.h / planet.cpp)

Stores all planet properties in a structured way.

```
struct Planet {
    string name;
    float radius;           // sphere size
    float orbitRadius;      // distance from Sun
    float orbitSpeed;       // revolution speed
    float rotationSpeed;    // self-rotation speed
    float orbitAngle;       // current angle in orbit
    float rotationAngle;    // current self-rotation angle
    GLuint textureID;       // loaded texture
    float r, g, b;          // fallback color
};
```

Module 2: Renderer (renderer.cpp)

- drawSun() — renders glowing sun sphere
- drawPlanet(Planet p) — renders a planet with texture + shading
- drawOrbit(float radius) — draws circular orbit path
- drawStars() — renders star background
- drawSaturnRing() — flat disc around Saturn
- drawAsteroidBelt() — small random spheres between Mars & Jupiter

Module 3: Camera (camera.cpp)

- Stores camera position, target, up vector
- Mouse drag → rotate camera around origin
- Scroll → zoom in/out
- R key → reset to default view

Module 4: Lighting (lighting.cpp)

- GL_LIGHT0 placed at Sun's position (0, 0, 0)
- Ambient light: low (space is dark)
- Diffuse light: high (Sun is bright)
- Specular light: medium (shiny planet surfaces)

Module 5: Texture Loader (texture.cpp)

- Uses stb_image.h to load JPG/PNG textures
- Returns GLuint texture ID
- Applied using glBindTexture + glTexImage2D

Module 6: UI (ui.cpp)

- Planet name labels using glutBitmapCharacter
- Info panel: name, distance, orbital period
- FPS counter
- Controls help text

11. Tech Stack & Tools

Tool	Purpose
C++	Main programming language
OpenGL	Core graphics rendering API
GLUT / GLFW	Window creation & input handling
stb_image.h	Texture/image loading (header-only)
GLU	Sphere rendering (gluSphere)
VS Code / Code::Blocks	IDE
g++ (GCC)	Compiler

Installation (Linux)

```
sudo apt-get install freeglut3-dev
sudo apt-get install libgl1-mesa-dev
sudo apt-get install libglu1-mesa-dev
```

Compile Command

```
g++ main.cpp src/*.cpp -o solar_system -lGL -lGLU -lglut -std=c++17
```

12. Controls & Interaction

Key / Action	Function
Mouse Drag (Left Button)	Rotate camera around solar system
Scroll Up / W	Zoom In
Scroll Down / S	Zoom Out
SPACE	Pause / Resume animation
+ / -	Increase / Decrease simulation speed
O	Toggle orbit lines ON/OFF
L	Toggle planet labels ON/OFF
R	Reset camera to default
Click on Planet	Show planet info panel
ESC	Exit application

13. Expected Output

Visual Output

- A dark space background filled with stars
- Bright glowing Sun at the center
- 8 planets orbiting the Sun at different speeds and distances
- Each planet has a real texture (Earth looks like Earth, Jupiter has bands, etc.)
- Saturn has a visible ring
- Moon orbits Earth
- Asteroid belt visible between Mars and Jupiter
- Smooth 60 FPS animation

Academic Output

- Demonstrates mastery of full OpenGL pipeline
- Covers 10+ CG course topics in one project
- Clean, modular, well-documented code
- Interactive — professor can explore during viva
- Visually impressive for presentation

14. References

1. OpenGL Programming Guide (Red Book) — 8th Edition
2. Computer Graphics with OpenGL — Donald Hearn & M. Pauline Baker
3. learnopengl.com — Modern OpenGL tutorials
4. NASA Solar System Exploration — <https://solarsystem.nasa.gov>
5. Planet Textures — <https://www.solarsystemscope.com/textures/>
6. stb_image.h — <https://github.com/nothings/stb>

Summary

This project is a **complete, real-world inspired, visually rich 3D simulation** that covers every major topic of the Computer Graphics course. It is unique because it combines scientific accuracy, interactive exploration, and full CG pipeline implementation in a single impressive application. With a clean 2-member work division and a 6-week timeline, this project is perfectly achievable and guaranteed to impress.

Documentation prepared for CSE Department, 7th Semester, USTC
Course: Computer Graphics and Image Processing